152 自然言語処理支援エディタの改良

堤 正勝† 荒木 健治†† 宮永 喜一† 栃内 香次†
†北海道大学工学部 ††北海学園大学工学部

1 はじめに

昨年の支部大会報告 [1] で述べているように、自然言語処理の研究を行なうには、ある程度の操作環境が提供されることが必要である。著者らはこのため、まずその環境の最も基礎的な部分を、文書の形態素解析を半自動的に行なうエディタという形で構築している。本稿では、昨年報告した内容にさらに追加した機能などを報告したい。

2 改良の要点

本エディタの特徴は、通常のエディタでは文書は文字単位あるいは行単位で管理されているのに対して、単語を中心に管理している点にある。さらに、後述するように、単語以外の情報も管理できる。本エディタの以前のバージョンでは一つのプログラムで一つの文書を扱い、その際に、単語辞書もそのたびにロードする形式をとっていた。しかし、通常のかな漢字変換がそうであるように、複数のテキストに対して、主記憶にロードされる辞書はひとつでよい。そこで、辞書はロードしたままで、複数のテキストを扱うようにした。また、編集時のキー入力も単一の定義を用いたため、すぐにマルチストロークの入力が必要になり、作業効率の低下が目に見えていた。そこでより少ないキー入力で操作できる方法を導入した。

3 バッファおよびモードの定義

昨年の段階では、単一のファイルを単語のリストである行のリストとして管理していた。このリストをファイル管理情報とともに一つのバッファ領域に収め、これを複数用意する。情報の柔軟性を考慮し、単語や行と同様にリスト型で管理している。バッファは、主記憶の許す限りいくらでもリストに登録できる。

次にキー入力の問題を考える。入力キーに追加する割り当ては一般に特殊な機能であることが多い。このためキー割り当てにはスコープが存在するのが好ましい。そこで、編集モードを定義し、編集モード毎にキー定義のマッピングを行なうのが良いと思われる。マッピング情報は、より基礎的なモードのマップを継承できるように考慮する。

このような形でバッファとモードを定義すると、モード毎に異なる編集形態を提供できる。そこで、以前のエディタにおける表示管理、入出力、キーボード入力部分をモード情報にまとめて管理する。具体的には モード名 (name), 初期化 (init), 終了 (quit), 描画 (redraw), キー入力 (keyin) を一つの構造体として管理する。バッファの開設に伴い、init() を呼びだし、バッファの消去に伴い、quit() を呼び出すようにする。init() はファイルをリスト構造に展開する。モードによっては形態素解析を行なう。redraw() はキーボード入力と、端末からの信号 (仮想端末のサイズ変更など) で呼び出され、keyin() は文字通りキーボード入力で起動する。モードはプログラムの起動時に用意され、バッファの開設要求とともにに指定する。新しいモジュールを作成するときは、この4つの関数をインプリメントすることで作成できる。

4 辞書バッファ

このような形でバッファを定義すると、辞書管理バッファも同様に管理できる。redraw() で入力された単語を検索し、同音語リストを表示する画面を用意して、同音語の追加、削除、優先順位変更などの編集機能を keyin() で提供する。なお、このモジュールはすでにインプリメントした。辞書バッファは行でなく、木構造で管理しているが、モードの関数と、そのサブルーチン群がすべて管理している。init() では、辞書ファイルを単語構造体に展開し、二分木に登録する。quit() は辞書が編集されていれば、辞書ファイルを更新する。単語の木構造中の検索や登録は他のモジュールからも呼び出せる。

5 ノードについて

以前のバージョンでは行や単語を管理するために、直接、行や単語に再帰定義のポインタを使用せず、ノードを用意して構造体を指し示す形にしていた。行はノード同士の双方向リストで、ノードの内容も単語リストを示すノードへのポインタであった。また単語リストは単語を指すノードの双方向リストであった。今回の改良で、ノードのメンバにノードタイプ識別子を追加し、多様な情報を指し示すことができるようにした。バッファのリストもこのノードで接続されている。テキストはノードの連結で表現されており、ノードに多様な種類があるので、例えば、そのテキストの単語リスト中に、突然、バッファを指すノードが現れることが許される。現在プロトタイプ版のインプリメントの終了したテキストモードモジュールの redraw() では、表示の時には色を変えてバッファ名を表示する。バッファ名のところで特定のキーを叩くと、バッファ名は編集されないので、そのバッファの内容の編集モードに画面がスイッチするようにしてある。これはハイパーテキストと同様である。

6 マッチングアルゴリズムについて

本エディタでは、仮名漢字変換や、形態素解析など、パターンマッチを頻繁に使用する。しかし、辞書バッファがそうであるように木構造を逐一文字列のマッチングをとって検索していては、大語彙を扱う上で不利である。そこで、検索を行なう前に、動的メモリ上に配列をとり、検索の効率をあげる手法を考えている。具体的には、ソーティングされた辞書は一般に部分文字列が同じものが多いことに注目し、候補範囲を狭めてゆく手法であるが、候補範囲を示すために状態 ID[] を用意して ID[n] と次の文字 C から ID[n+1] を一意に決定し ID の終端フラグによりマッチング成功とする。このような手法は検索前に ID に関する情報を作成する（辞書のコンパイル）必要があり、実験プログラムを作成し、効率の点で検討している。

7 形態素に関する追加

形容詞の管理は最後が k で終る形で管理していた。

例) hayak-i
hayak-ar-ou

学校文法でいわれるところの語幹を中心に品詞の転用が行なわれるため、語幹登録を行ない、単語管理の表示処理で k を付加するようにした。

品詞の転成に関しては次のようなものがある。

haya-k-ar-qt-a - 通常形容詞
haya-m-er-u - 動詞化
atata-kak-i - 通常の形容詞
atata-kada - 形容動詞化
atata-m-er-u - 動詞化

8 おわりに

本稿では、昨年より作成中のエディタの機能追加（改良）について述べた。現在、モードのレパートリを増やす他、上記のパターンマッチアルゴリズムの検討、また形態素に関する情報の収集などを行なっている。

参考文献

[1] 堤 正勝、荒木健治、宮永喜一、栃内香次：自然言語処理支援エディタの作成平成5年度電気関係学会北海道支部連合大会，北大 (1993).